4주차

자바웹프로그래밍(2)

강사 : 최도현



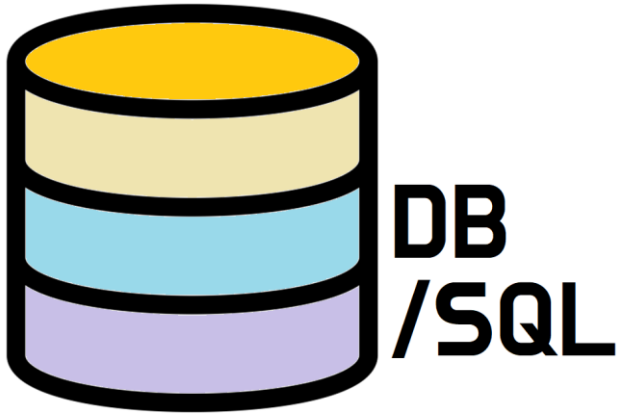
오늘의 할일 - 1

트렌드

- 웹 트렌드 분석

데이터베이스

MYSQL

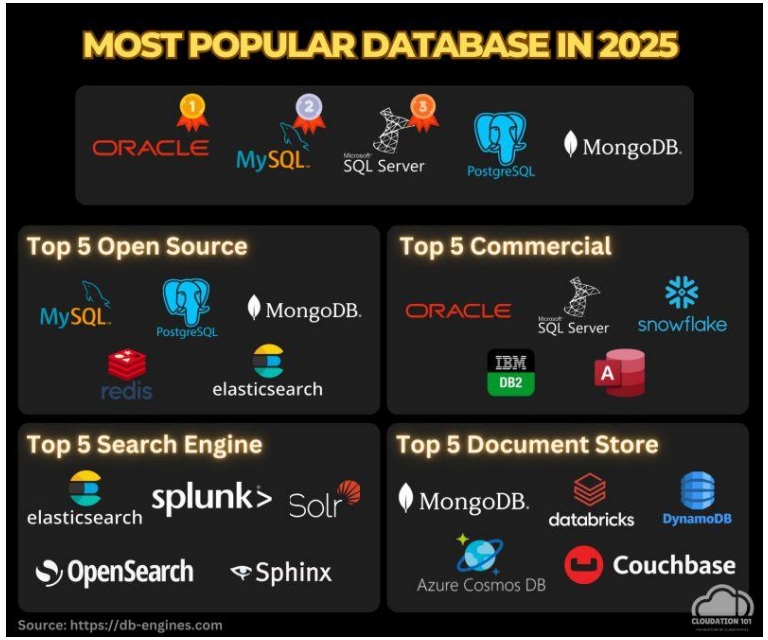


트렌드

데이터베이스

- ORACLE, MYSQL 아직 RDB 대세
  - PostgreSQL, MariaDB, SQLite 등
- 오픈소스 DBMS 인기
  - 클라우드 서버 고도화
    - 클라우드·컨테이너 환경 확산
    - 사이버 보안, AI/ML
  - 현재 RDBMS 주류
- 최신 AI 및 빅데이터
  - NoSQL 등 시장 크게 성장





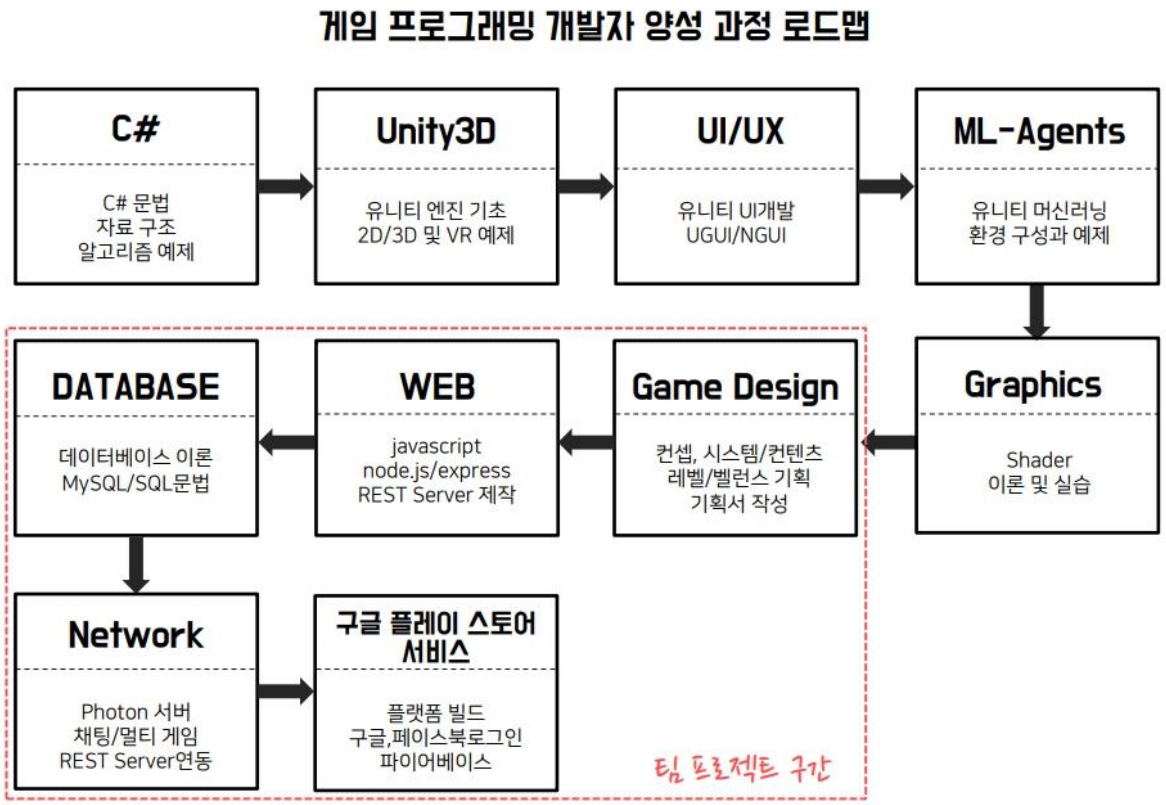
데이터베이스

- 프로젝트 시작 단계
  - 데이터베이스 선택, 비용!!

- 게임 서버? : 안정성/호환성
  - 최적화 VS 플랫폼 종속성
    - 유니티? SQLite, Mysql 등

  - 예) 장르 별 게임 환경
    - 속도/일관성/지연, 확장성 등
    - 게임의 사양/엔진
    - 접속자 규모 변화

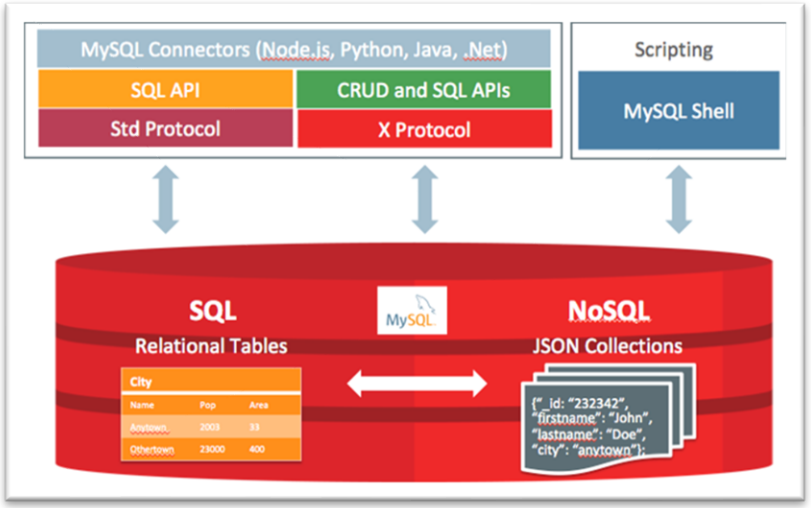


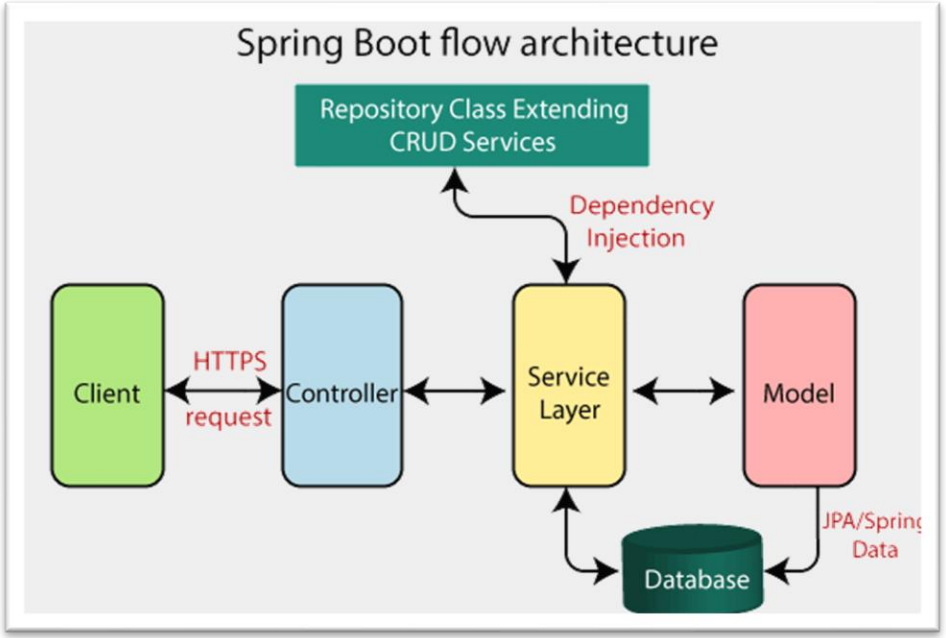


트렌드

MYSQL

- 오픈 소스 기반 RDB, 공식 웹사이트
  - 2010년, 오라클이 썬 인수
    - 무료/상용화 버전 분리
    - 최신 8.X~9버전, NOSQL 지원 시작
  - 오픈 코어 모델 도입
    - 소스 오프 BUT, 부가 기능 유료화
    - MySQL Enterprise 버전
    - 백업, 모니터링, 고가용성 등 고급 기능 보강
  - 대부분 게시판, 블로그, CMS 등에서 활용
    - 대표 : 워드프레스, 리눅스 기본(APM)
- **스프링 부트 4.x 환경**
  - JPA를 통한 데이터 제어 지원



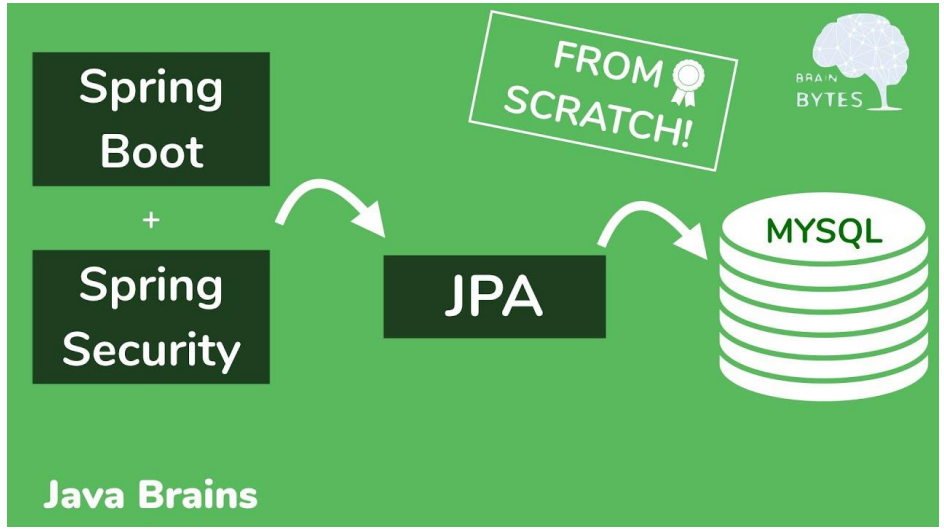


오늘의 할일 - 2

- 포트폴리오 작성하기

프로필 수정하기(추가)

데이터베이스 연동



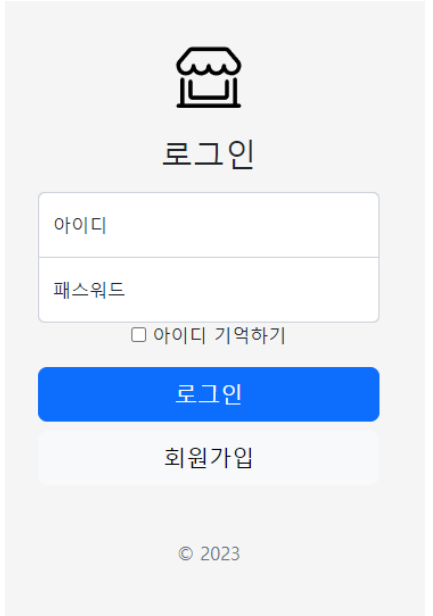
2학기 개발 요약

- 메인 화면(매주)
  - (PUBLIC)개인 포트폴리오 화면 추가
    - 부트스트랩5 기반 디자인 적용

- **데이터베이스 연동**
  - 로그인/로그아웃
  - 이미지 업로드/페이징
  - 게시판 CRUD
  - 주문/장바구니
  - 실시간 알림
  - 예약/중복 방지





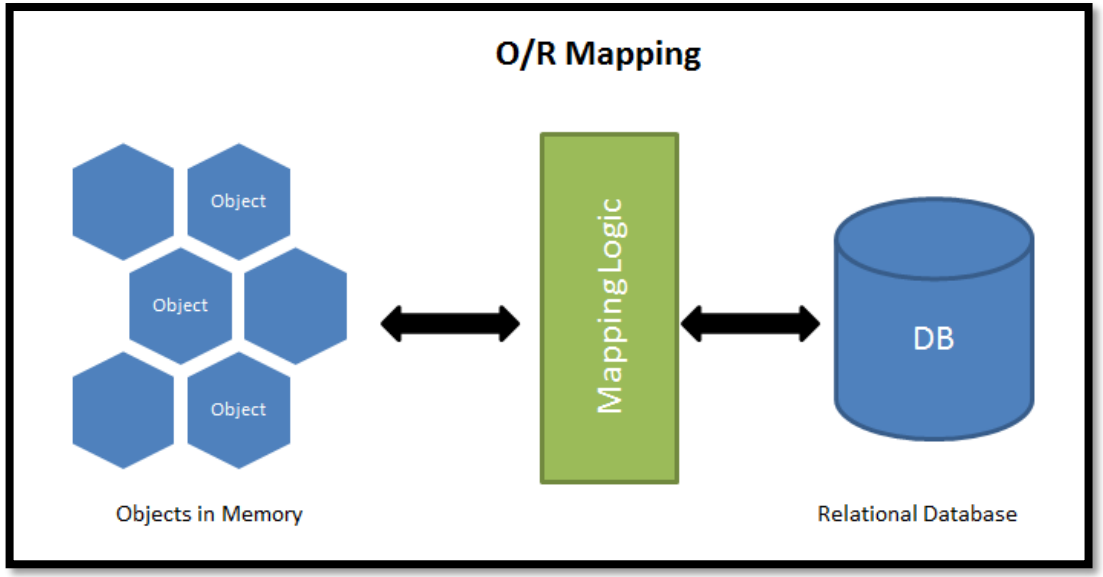


실습

데이터베이스 연동

- 프로젝트 최상위 폴더
  - pom.xml 설정 파일을 연다.
    - Spring Data JPA 모듈 XML 태그 확인
    - 지난주에 주석처리함
- JPA(Java Persistence API)
  - ORM : OOP 객체(클래스) 매핑 역할
    - SQL을 JPA가 대신 생성
    - 데이터 조회, 수정, 생성, 삭제
  - 쿼리가 아닌 메서드로 DB 제어
    - 자바 코드로 제어
- 왜요? 기존 데이터 제어 구현
  - 개발자마다 다른 방식? 표준화

```
<dependencies> Add Spring Boot Starters...
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web-services</artifactId>
    </dependency>
```

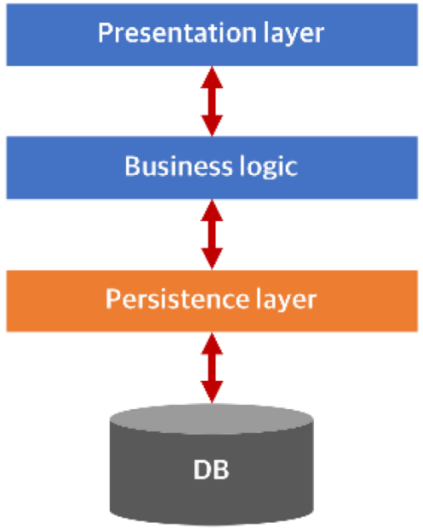


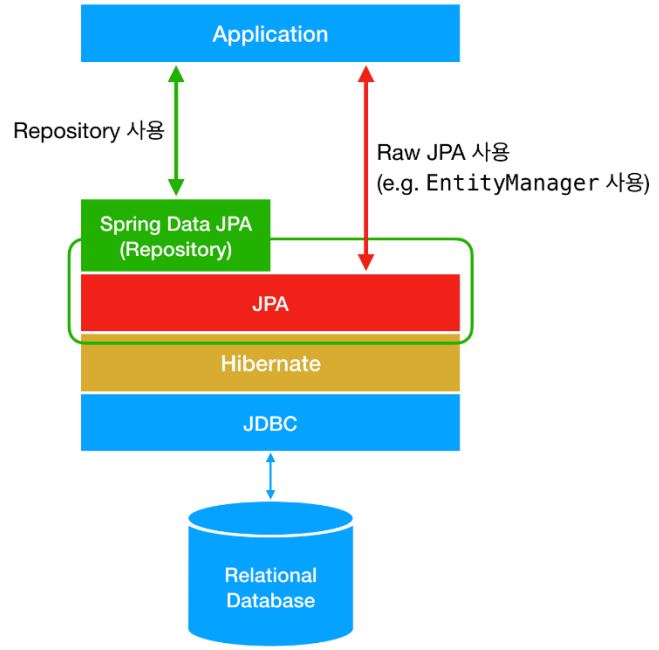
데이터베이스 연동

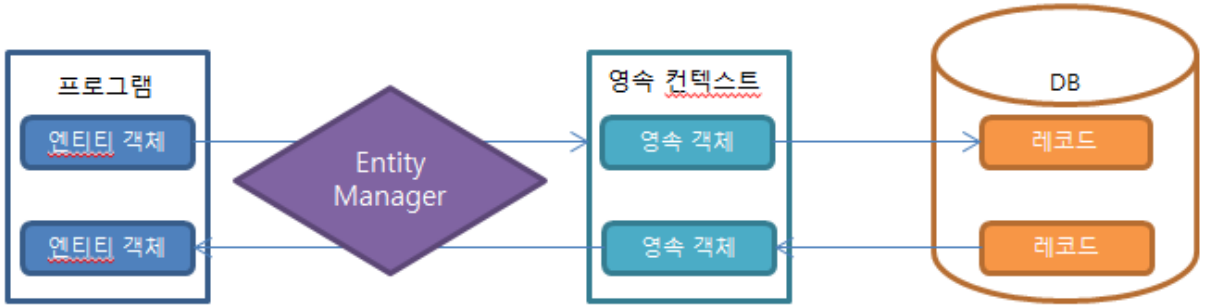
- 데이터 제어와 영속성 제공
  - 영속성 : 종료 이후 DB 저장
  - 영속 계층(Persistence)에 위치

- Repository 내부의 엔티티 매니저
  - 영속성 컨텍스트 자동 관리
  - 트랜잭션 처리의 복잡성 감소

- JPA의 대표 구현체(ORM)
  - 하이버네이트





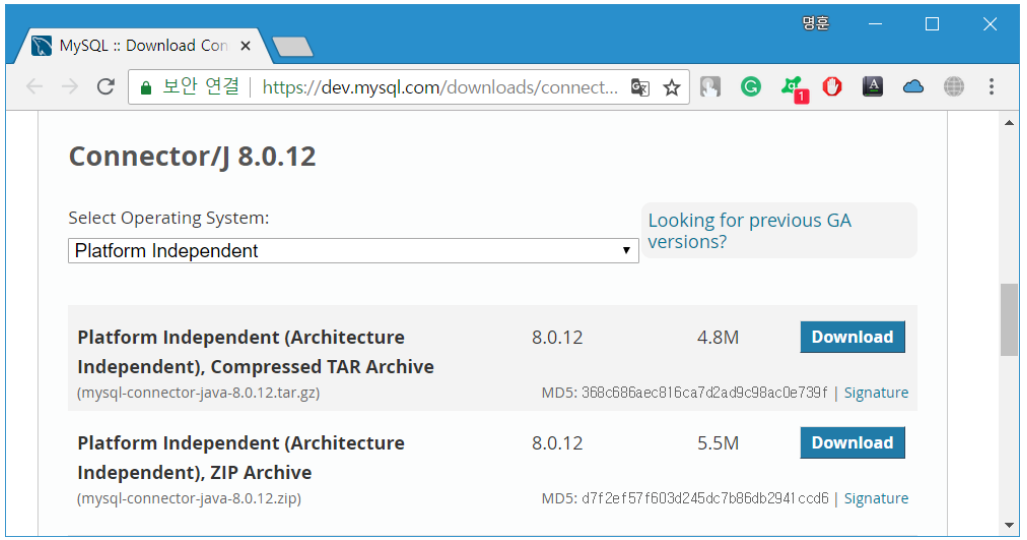


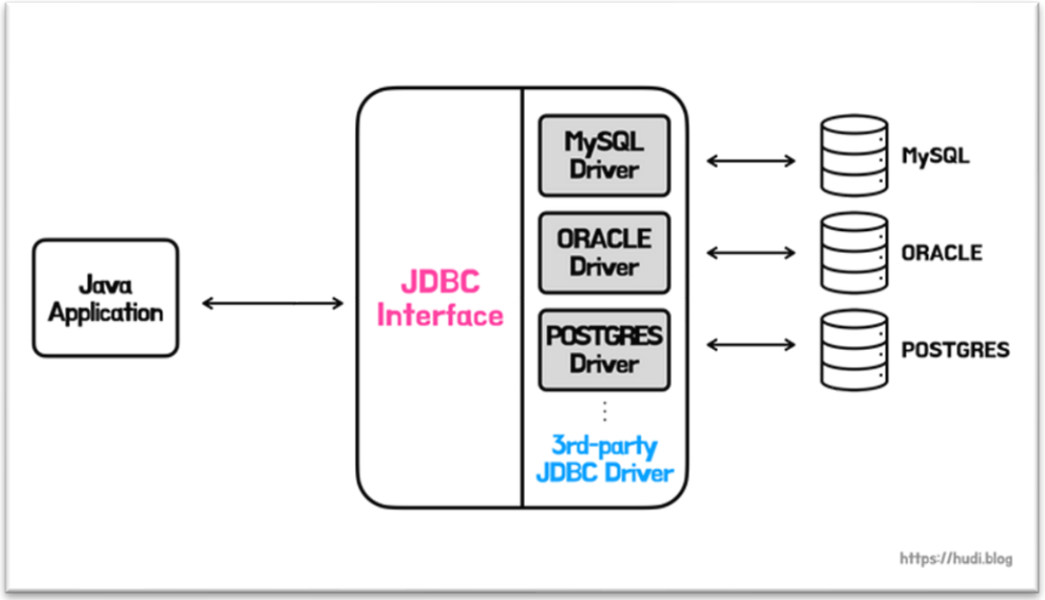
실습

데이터베이스 연동

- 프로젝트 최상위 폴더
  - pom.xml 설정 파일을 다시 연다.
  - MYSQL 커넥터 확인
    - JPA에 MYSQL 연결 드라이버 포함 X
      - 참고 : 스프링 3 버전 이후 이름 j로 변경됨
  - 커넥터 설정 없는 경우 직접 추가

```
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <scope>runtime</scope>
    </dependency>
```

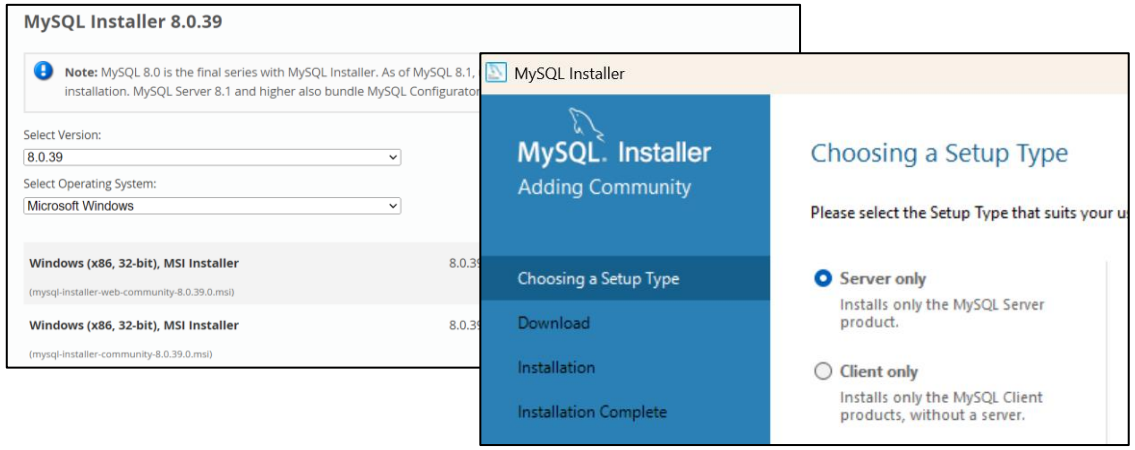


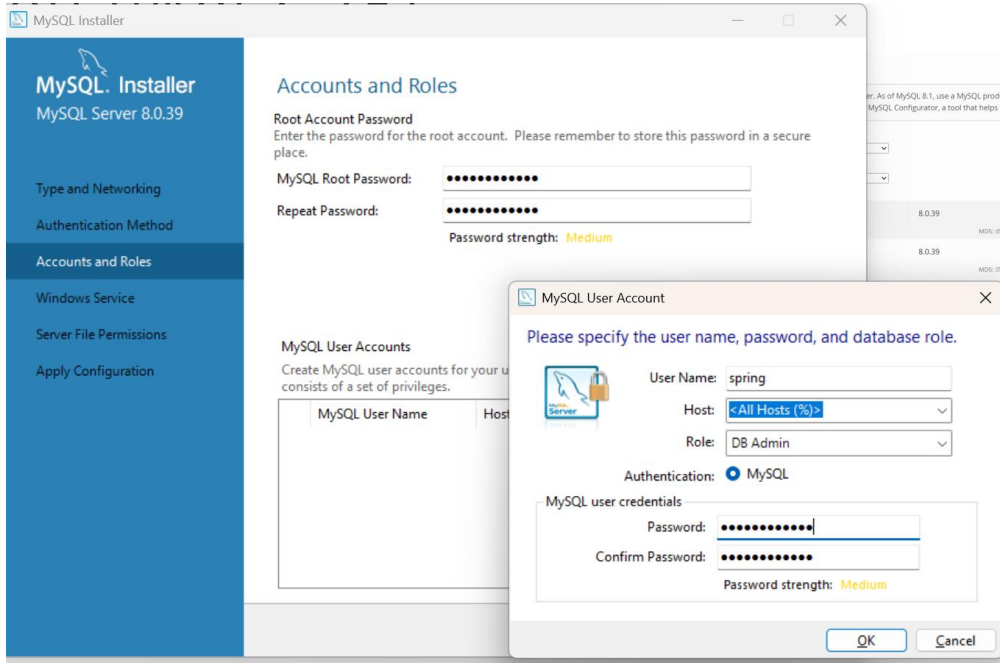


실습

데이터베이스 연동

- MYSQL 다운로드 및 설치
  - 공식 웹사이트, 최신 8버전
    - 윈도우 303Mb, 설치 버전 다운로드
    - 맥·리눅스 사용자는 Docker 설치
  - Server only로 설치를 진행
    - 기본 설정으로 next 진행
  - 계정 관련
    - root 관리자 계정, 패스워드 2회 입력
    - spring, 패스워드 2회 입력
  - 설치를 완료한다.



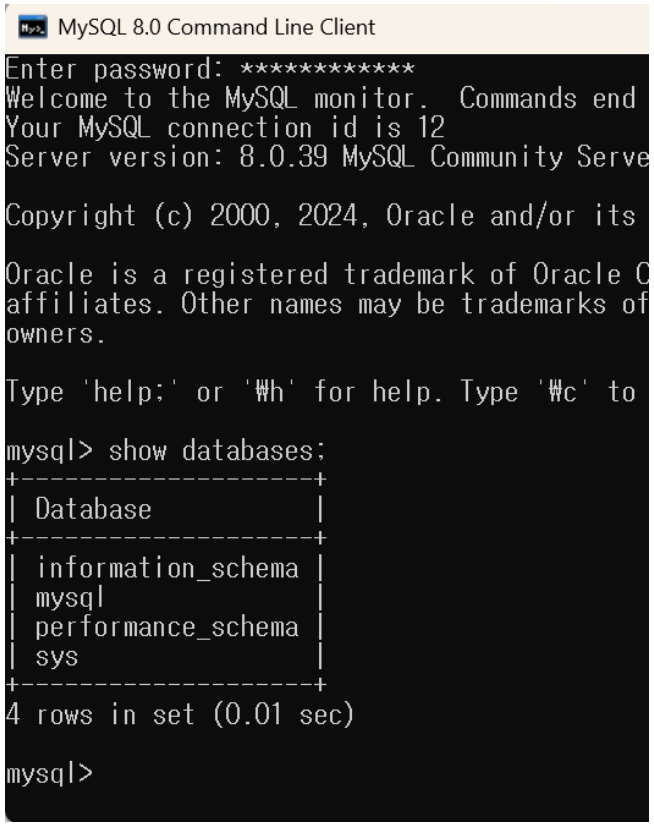


실습

데이터베이스 연동

- MYSQL 접속 및 데이터베이스 생성
  - Command Line Client 실행하기
    - 시작 → mysql → 실행
  - root 패스워드 입력
    - 접속 후 데이터베이스 확인
    - show databases;
      - 기본 db가 생성되어 있다.
  - 데이터베이스 생성
    - create database spring;
    - flush privileges;





실습

데이터베이스 연동

- MYSQL 접속 정보 추가하기
  - application.properties 설정 파일 연다.
    - 다양한 공통 애플리케이션 속성



  - 데이터베이스 계정 정보 추가
    - MYSQL 서버 기본 포트 : 3306

```
spring.datasource.url=jdbc:mysql://localhost:3306/spring?serverTimezone=Asia/Seoul
spring.datasource.username=root
spring.datasource.password=패스워드
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

  - 프로젝트를 실행해보자.
    - 터미널 : 정상 동작 확인

		Spring Boot 프로젝트
의존성 관리	직접 관리	Spring Boot 스타터 라이브러리 사용
설정	XML 파일 또는 Java 클래스	`application.properties` 파일
실행	`main()` 메서드가 있는 클래스	`SpringApplication.run()` 메서드
기타 차이점	더 많은 구성 옵션, 더 많은 유연성	더 간단한 설정, 더 빠른 개발 속도

Sheets로 내보내기

실습

데이터베이스 연동

- MYSQL 정상 접속 화면
  - HikariPool의 Start Completed
    - 기본 커넥션 Pool, 연결 최적화
    - 첫번째 그림 : 정상
  - 정상 실행이 안되는 경우
    - 터미널에서 확인
      - 예) 패스워드, db 오류
    - Pom.xml에 주석 해제
      - 2가지 모듈 해제안한 경우

```
Initializing Spring embedded WebApplicationContext
Root WebApplicationContext: initialization completed in 4321 ms
HHH000204: Processing PersistenceUnitInfo [name: default]
HHH000412: Hibernate ORM core version 6.5.2.Final
HHH000026: Second-level cache disabled
No LoadTimeWeaver setup: ignoring JPA class transformer
HikariPool-1 - Starting...
HikariPool-1 - Added connection com.mysql.cj.jdbc.ConnectionImpl@7b43

HikariPool-1 - Start completed.
HHH000489: No JTA platform available (set 'hibernate.transaction.jta.

Initialized JPA EntityManagerFactory for persistence unit 'default'
spring.jpa.open-in-view is enabled by default. Therefore, database qu
```

```
java.lang.NullPointerException: Cannot invoke "org.hibernate.engine.jdbc.spi
because the return value of "org.hibernate.resource.transaction.backend.jdbc
l
        at org.hibernate.resource.transaction.backend.jdbc.internal.JdbcIsol
[hibernate-core-6.5.2.Final.jar:6.5.2.Final]
        at org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator.g
java:290) ~[hibernate-core-6.5.2.Final.jar:6.5.2.Final]
        at org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator.i
```

```
        ... 18 common frames omitted
Caused by: org.hibernate.HibernateException: Unable to determine Dialect without JDBC
' for common cases or 'hibernate.dialect' when a custom Dialect implementation must be
        at org.hibernate.engine.jdbc.dialect.internal.DialectFactoryImpl.determineDial
6.5.2.Final.jar:6.5.2.Final]
        at org.hibernate.engine.jdbc.dialect.internal.DialectFactoryImpl.buildDialect(
.Final.jar:6.5.2.Final]
```

실습

데이터베이스 연동

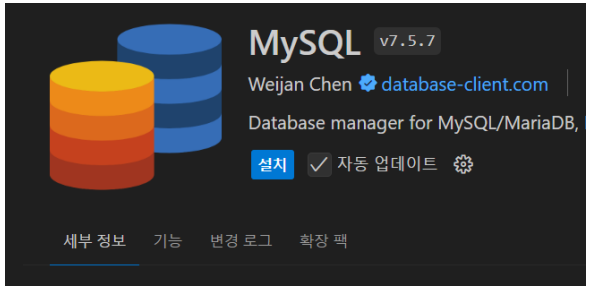
- 코드 확장 모듈
  - 데이터베이스 쉽게 확인하자.

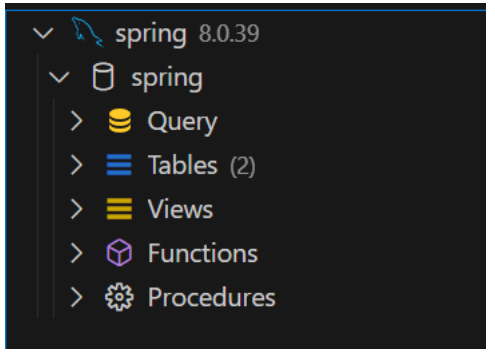
  - 확장 : mysql 검색 후 설치
    - 설치 완료 후 최하단 MYSQL 확인

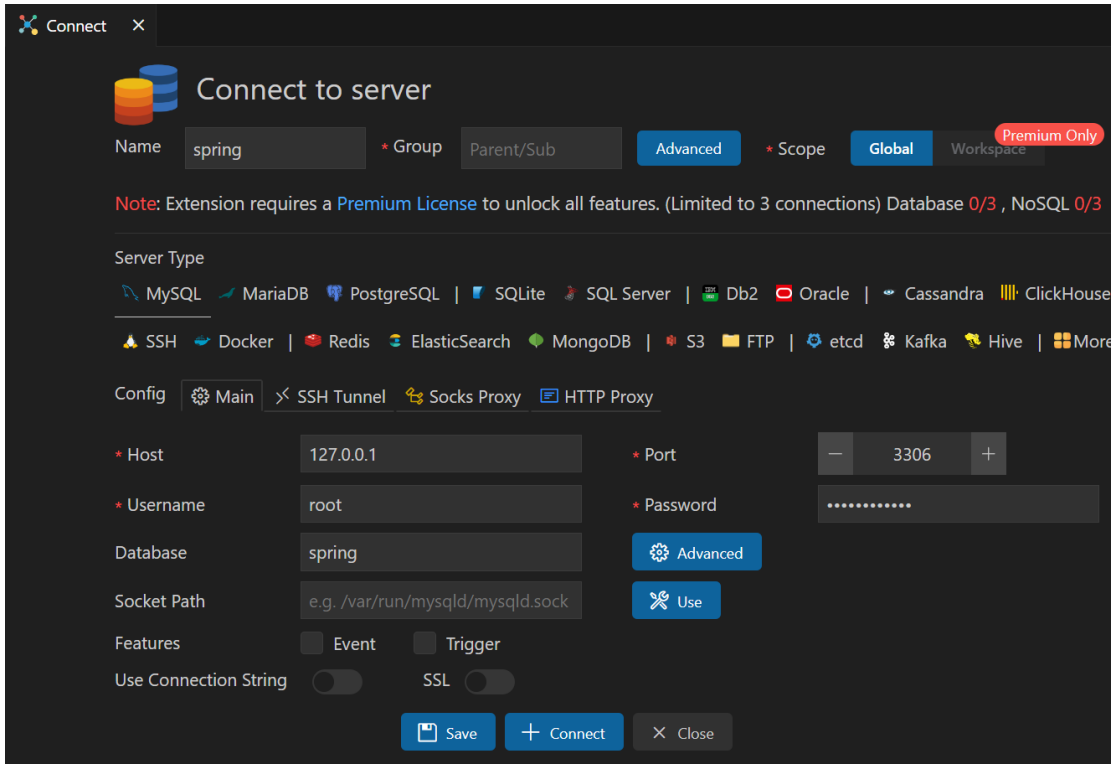
    - 좌측 Database 버튼 활성화됨
      - 클릭 후 필요한 정보 입력

    - 데이터베이스 이름 : spring

  - 하단 SAVE 후 연결 클릭
    - 빈 데이터베이스 확인!



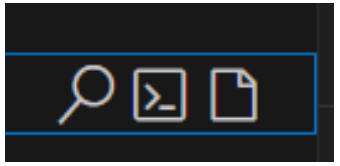


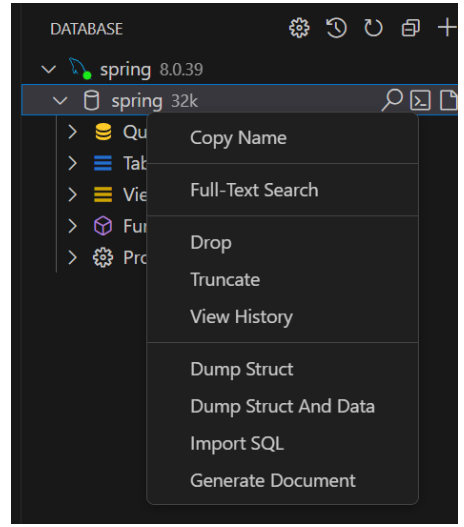


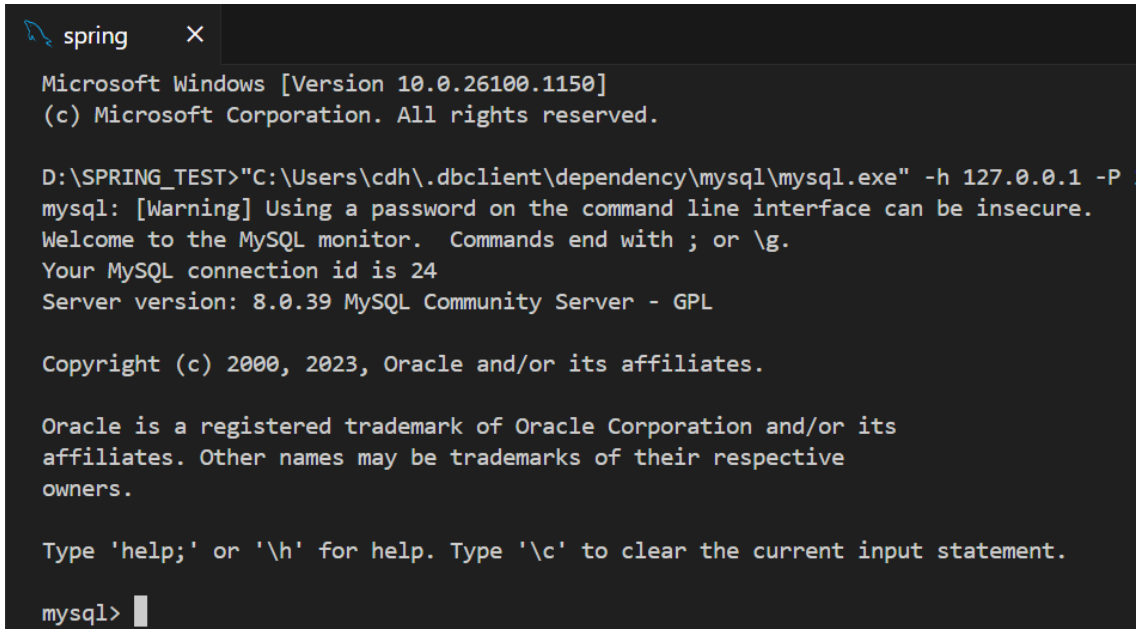
실습

데이터베이스 연동

- 기본 사용 방법
  - 확장 모듈 : 직접 확인
    - 서버 상태
    - dump, import 등 백업 가능
  - Open in terminal 클릭
    - 터미널 수행 가능
- 관리 툴로 계속 사용한다.

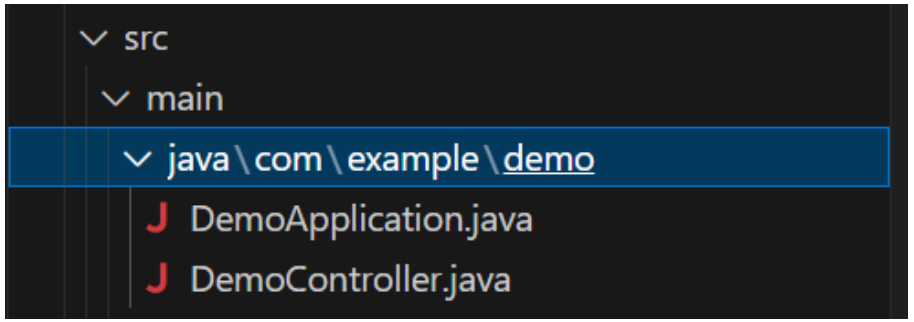




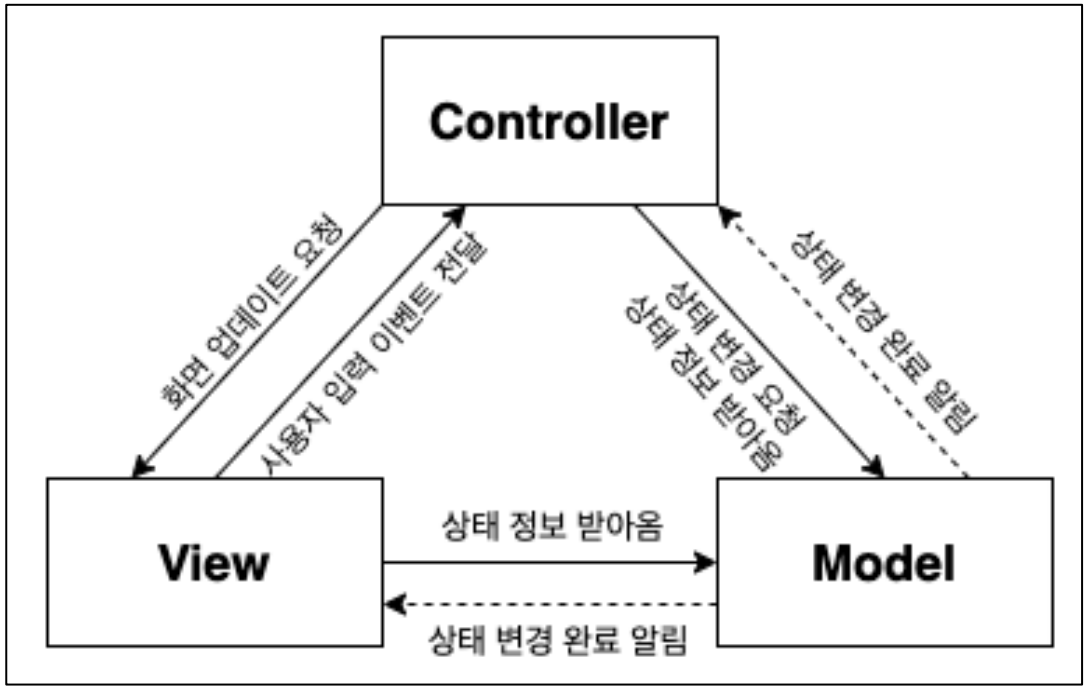


구조 변경

- 현재 우리 프로젝트 구조
  - 현재 동작 : 컨트롤러와 뷰 상호작용



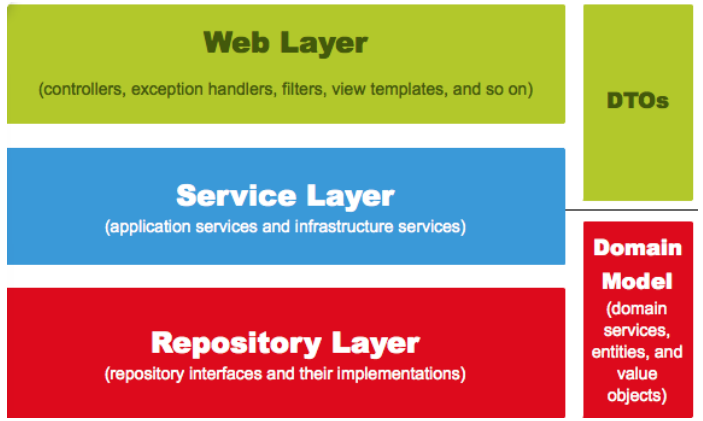
- 프로젝트 폴더 구조 확인
  - demo 폴더 기준
    - 문제점? 단일 폴더다.
  - 패키지 세분화의 필요
    - 또는 기능별로 분리
  - 잘 알려진 모델?
    - MVC 모델(기본) 등

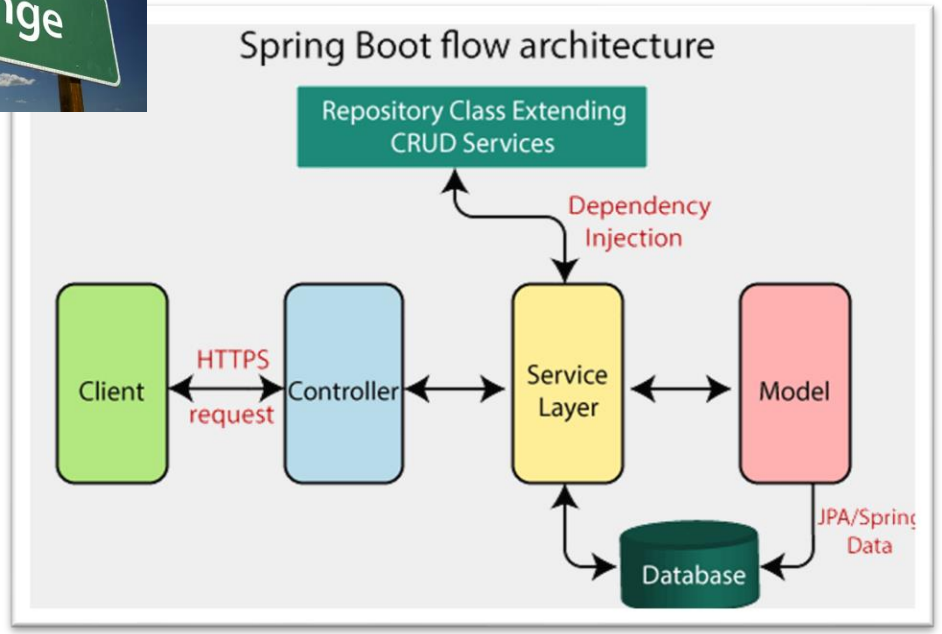


구조 변경

- 내부 구조 어떻게 분리/표현 할까?
  - **M : model 또는 domain 폴더**
    - 객체 클래스 – 엔티티
      - DB 테이블 구조
  - **V : view, 기존 templates 폴더**
    - 화면
  - **C : controller 폴더**
    - 요청/응답 흐름 제어/반환
  - **Service : service 폴더**
    - 중간 로직 처리(데이터 가공)
  - **Repository : repository 폴더**
    - DB 연결/접근(쿼리)



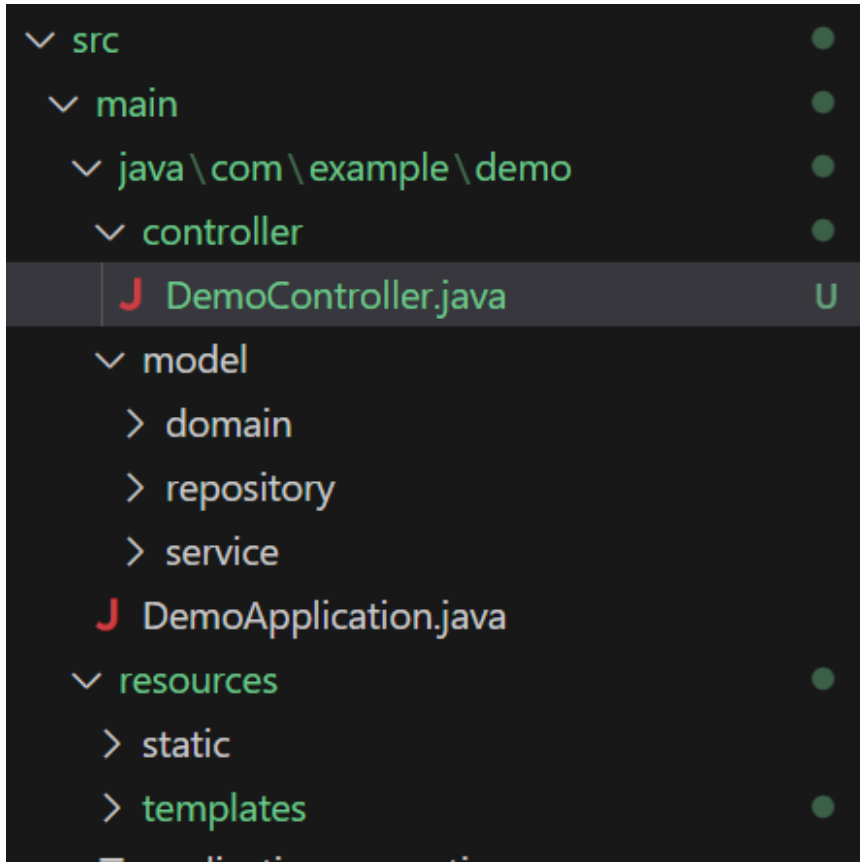




실습

구조 변경

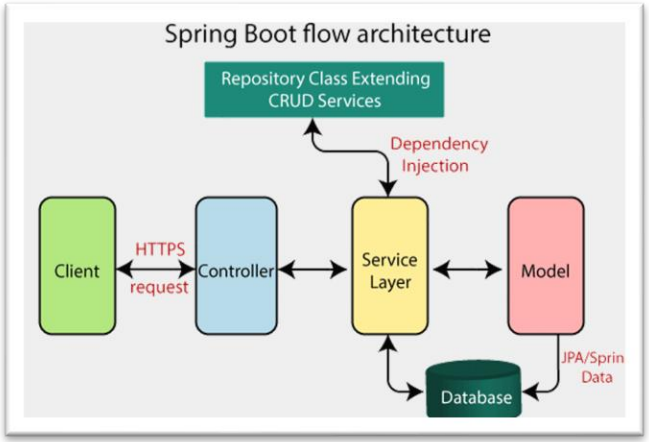
- 계층별 패키지 구조로 변경한다. (그림)
  - View : Front-end(변경 X)
    - templates 폴더 기존 유지
  - controller : controller
    - 기존 DemoController.java를 이동시켜 보자.
  - model
    - service, repository, domain 폴더 생성
  - 참고
    - DTO : 계층간 데이터 교환(보안)
      - 일반적으로 MODEL에 포함



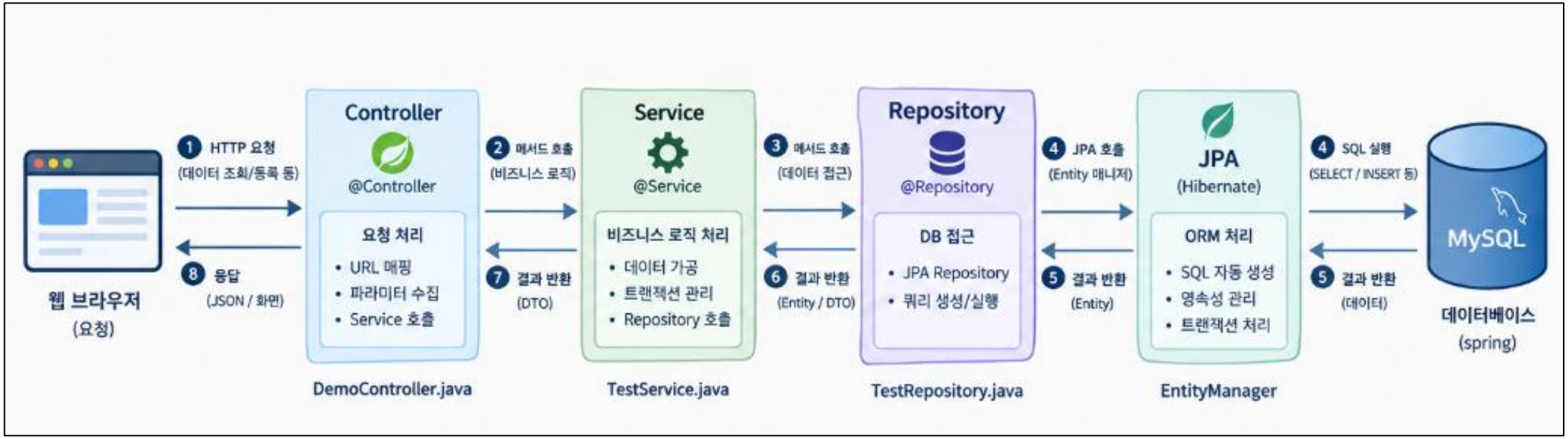
실습

구조 변경

- 기본 내부 구조와 실제 처리과정 비교
  - 핵심은 컨트롤러, 서비스, 리포지토리 이다.



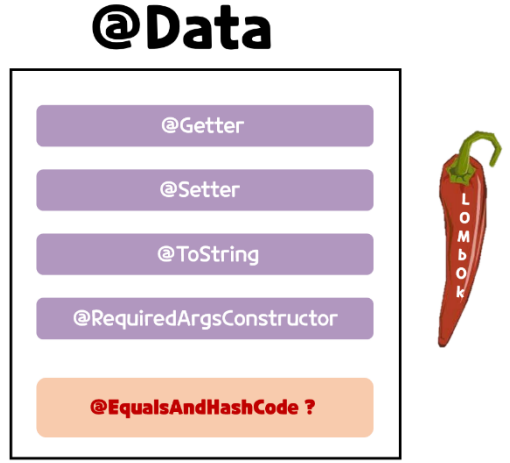
- 오늘 실습에서 소스코드 전체 흐름



실습

테스트 페이지

- 엔티티 클래스 추가 – Entity (영속 계층)
  - 데이터베이스 구조(테이블) 작성
    - domain 폴더에 TestDB.java 파일을 생성한다.




```
package com.example.demo.model.domain;

import jakarta.persistence.*;
import lombok.Data;

@Entity // TestDB라는 객체와 DB 테이블을 매핑. JPA가 관리
@Table(name = "testdb") // 테이블 이름은 testdb
@Data // set/get/tostring 등 필수 메서드 자동 생성
public class TestDB {
    @Id // 해당 변수가 PK
    @GeneratedValue(strategy = GenerationType.IDENTITY) // 값이 없어도 자동으로 할당
    private Long id;

    @Column(nullable = true) // 테이블의 컬럼 설정 값을 명시
    private String name;
}
```



실습

테스트 페이지

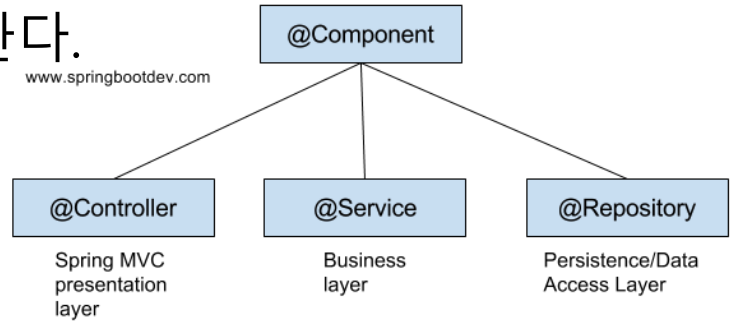
- 리포지토리 클래스 추가 – Repository (영속 계층)
  - 데이터베이스 제어 기능 작성
    - repository 폴더에 TestRepository.java 파일을 생성한다.

```
package com.example.demo.model.repository;

import org.springframework.data.jpa.repository.JpaRepository; // JPA 필수 등록
import org.springframework.stereotype.Repository; // 빈 등록
import com.example.demo.model.domain.TestDB; // 도메인 연동

@Repository // 리포지토리 등록
public interface TestRepository extends JpaRepository<TestDB, Long> {

    // Find all TestDB entities by a name
    TestDB findByName(String name);
}
```





  - 참고 : 리포지토리 등록?
    - jpa 상속받아 db 제어

실습

테스트 페이지

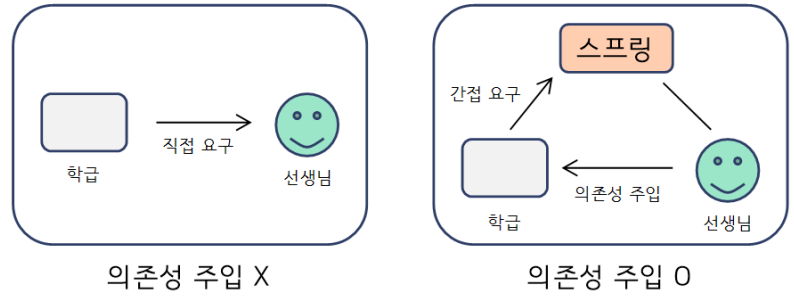
- 서비스 클래스 추가 – Service (서비스 계층)
  - 주요 기능(로직) 작성
    - service 폴더에 TestService.java 파일을 생성한다.

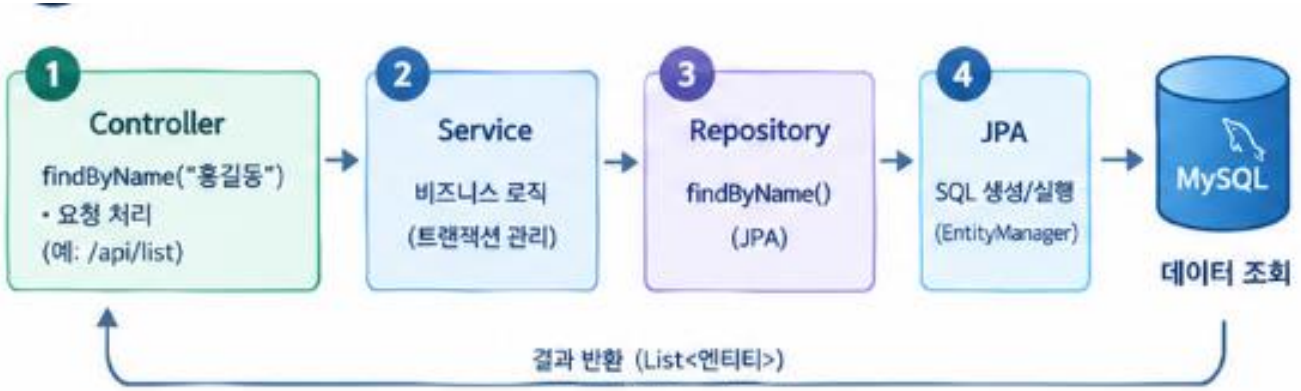
```
package com.example.demo.model.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import com.example.demo.model.domain.TestDB;
import com.example.demo.model.repository.TestRepository;

@Service // 서비스 등록, 자동 등록됨
public class TestService {
    @Autowired // 객체 의존성 주입 DI(컨테이너 내부 등록)
    private TestRepository testRepository;
    public TestDB findByName(String name) { // 이름 찾기
        return (TestDB) testRepository.findByName(name);
    }
}
```

(강한 결합 vs 약한 결합)





실습

테스트 페이지

- 컨트롤러 클래스 추가 – Controller (제어 계층)
  - 사용자 요청/응답 기능 작성
    - 기존 DemoController.java에 추가한다.




```
import com.example.demo.model.service.Testservice; // 최상단 서비스 클래스 연동 추가

// 클래스 하단 작성
@Autowired
Testservice testService; // DemoController 클래스 아래 객체 주입

// 하단에 맵핑 이어서 추가
    @GetMapping("/testdb")
    public String getAllTestDBs(Model model) {
        TestDB test = testService.findByName("홍길동");
        model.addAttribute("data4", test);
        System.out.println("데이터 출력 디버그 : " + test);
        return "testdb";
    }
```



  - 에러? 의존성 주입 등에 필요한 import 추가

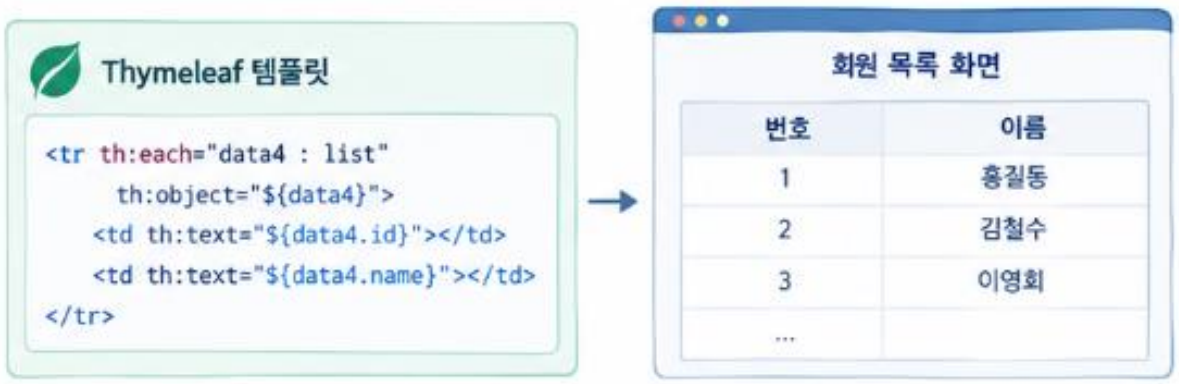
실습

테스트 페이지

- 뷰 템플릿, 프레젠테이션 - 화면(프리젠테이션 계층)
  - 화면 출력 기능 작성
    - Templates 폴더에 testdb.html을 생성한다.

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>데이터베이스 테스트</title>
</head>
<body>
    <h1>User List</h1>
    <p>아이디(기본키) : <span th:text="${data4.id}"></span></p>
    <p>이름 : <span th:text="${data4.name}"></span></p>
</body>
</html>
```

- 저장 후 서버를 정지/실행
  - 정상 실행 확인



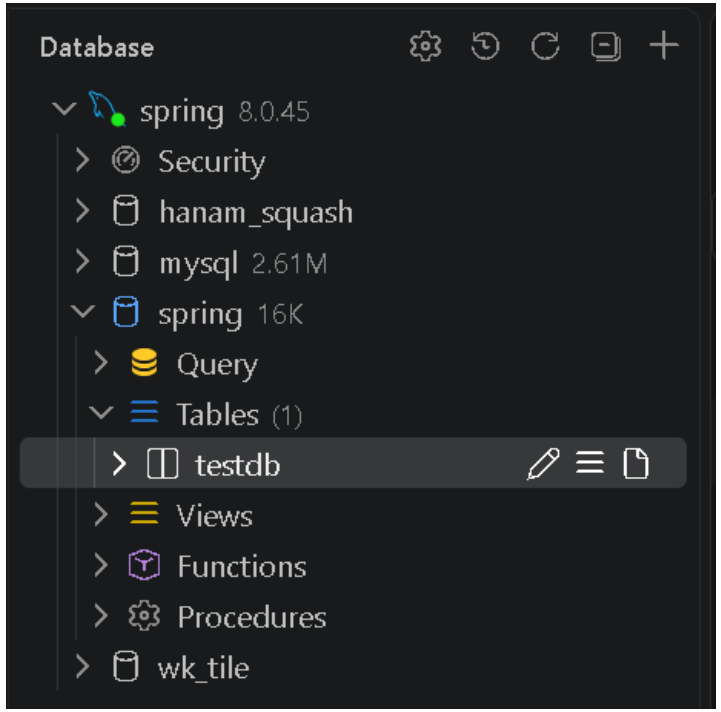
실습

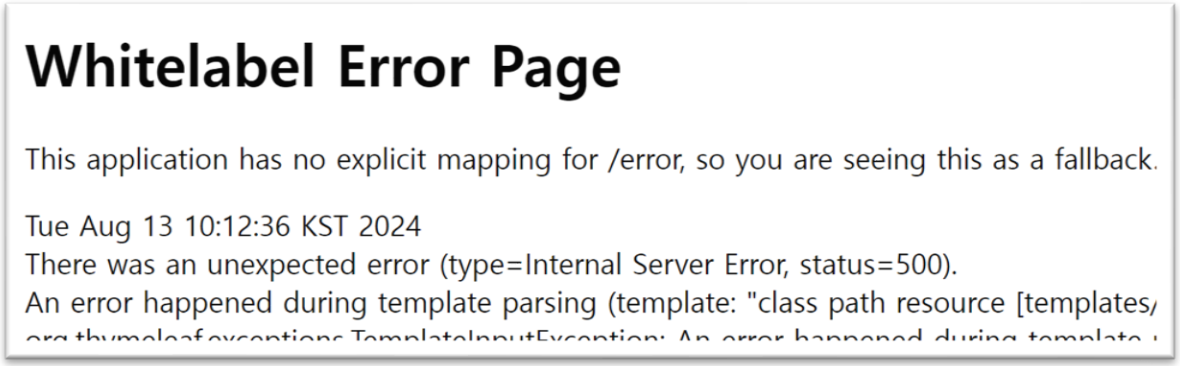
테스트 페이지

- 서버 재시작 이후 DB 확인
  - 내가 작성한 엔티티 클래스를 인식
    - 확장 기능에서 db 확인 : testdb 자동 생성됨
    - 테이블 객체와 db가 자동 맵핑됨

- localhost:8080/testdb 접속하기
  - 실제 내부 테이블 정보를 출력하는 화면
    - 데이터베이스는 OK

  - BUT, 데이터가 없다.
    - **빈 화면 또는 에러 페이지 출력**

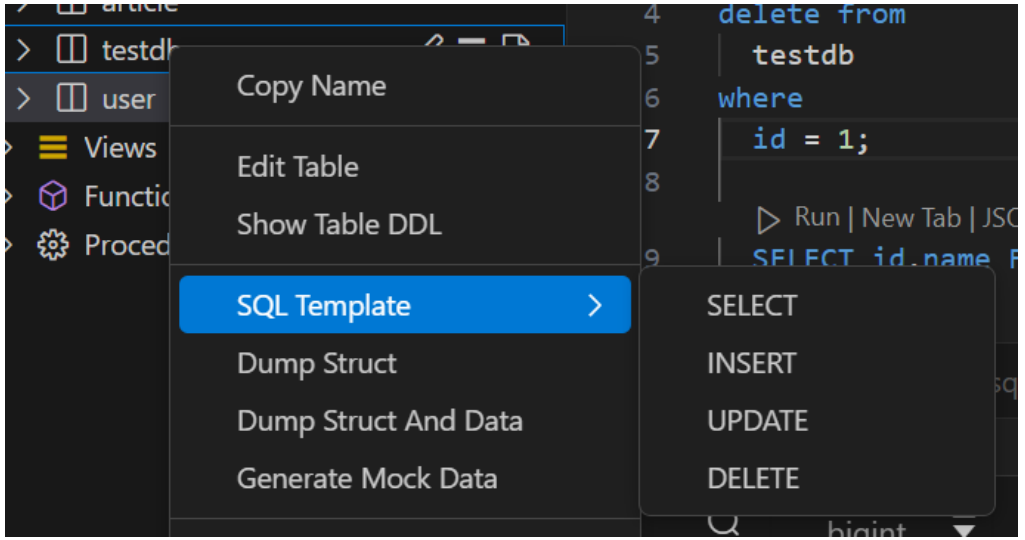


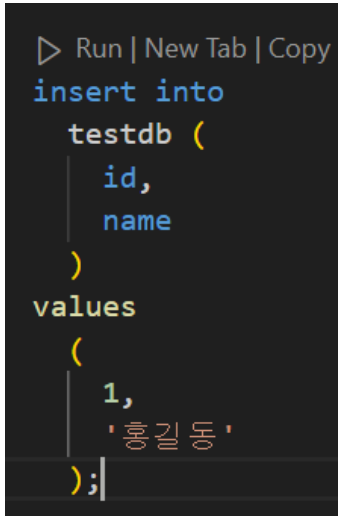


실습

테스트 페이지

- 직접 사용자 추가하기
  - testdb 오른쪽 클릭 이후
    - SQL Template → INSERT 클릭
    - 그림과 같이 수정 이후 Run 클릭
  - 사용자 추가 완료
- localhost:8080/testdb 접속하기
  - 입력된 키, 이름을 확인
    - 참고 : 새로 다시 조회, 새로고침 후 GO







실습

테스트 페이지

- 다수 사용자 출력하기
  - 기존 DemoController.java을 수정한다.
    - 참고 : 기존 코드 주석 처리
    - import 추가 연동

```
List<TestDB> users = testService.findAll();
model.addAttribute("users", users);
```

  - 기존 TestService.java을 수정한다.
    - 참고 : 기존 코드 주석 처리
    - import 추가 연동

```
public List<TestDB> findAll() {
    return testRepository.findAll();
}
```

JpaRepository 제공 메서드 목록

메서드	설명
findAll()	전체 조회 (SELECT * FROM testdb)
findAll(Sort sort)	정렬하여 전체 조회
findAll(Pageable pageable)	페이징 조회
findAllById(Iterable<ID> ids)	여러 PK로 조회
findById(ID id)	PK로 1건 조회, 없으면 Optional.empty()
getReferenceById(ID id)	PK로 프록시(지연 로딩) 참조
existsById(ID id)	PK 존재 여부
count()	전체 행 개수
save(S entity)	저장. PK가 없거나 새 엔티티면 INSERT, 있으면 UPDATE(merge)
saveAll(Iterable<S>)	여러 건 저장
saveAndFlush(S entity)	저장 후 즉시 DB에 반영(flush)
flush()	영속성 컨텍스트의 변경 내용을 DB에 반영
deleteById(ID id)	PK로 삭제
delete(T entity)	엔티티로 삭제
deleteAll()	전체 삭제
deleteAllById(Iterable<ID>)	여러 PK로 삭제
deleteAllInBatch()	한 번의 쿼리로 전체 삭제

실습

테스트 페이지

- **오늘의 문법 사용하기(매 주 업데이트)**
  - Templates 폴더에 testdb.html를 수정한다.
    - 기존 코드는 주석처리

```
<table border="1">
    <tr th:each="user : ${users}">
        <td th:text="${user.id}"></td>
        <td th:text="${user.name}"></td>
    </tr>
</table>
```

  - th:each로 tr을 반복 처리, 리스트(컬렉션) 속성 이름
  - 참고 : 공식 웹사이트
    - https://www.thymeleaf.org/



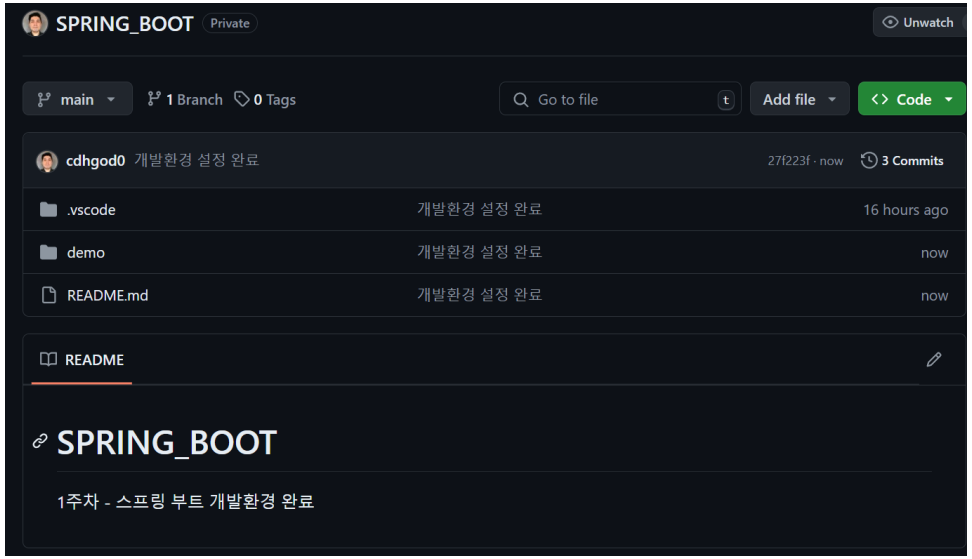
User List

1	홍길동
2	아저씨

잠깐! GIT 연동 – 소스코드 업로드

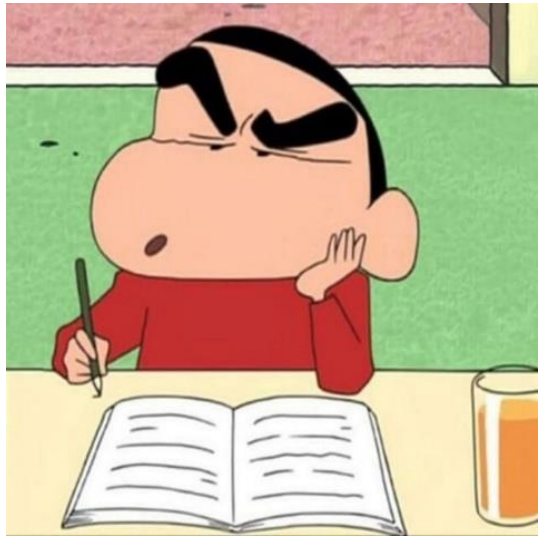
- README.MD 파일 열기 수정 예)
  - 4주차 데이터베이스 수정 완료
  - 추가된 파일 + 로 추가 → COMMIT (메인화면)
  - PUSH 하여 서버에 파일 업로드
- **앞으로 실습 코드를 계속 업데이트 하자.**
  - 폴더 또는 소스코드를 링크 가능하다.
  - 참고 : 깃 허브 소스코드 주소?
    - https://github.com/ 접속
      - 저장소 → 소스 파일 확인





•4주차 연습문제

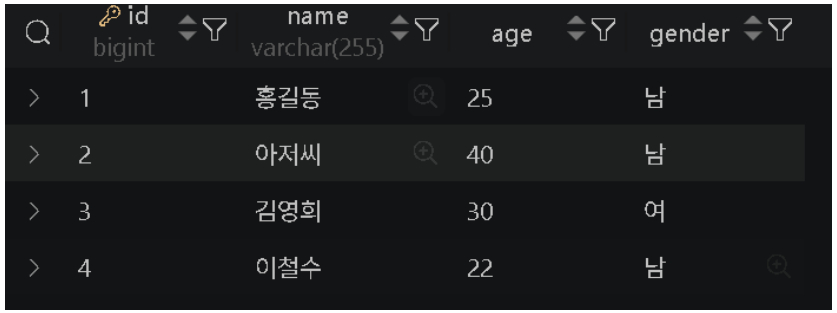
사용자 정보 수정 및 출력하기



문제

사용자 추가 및 출력하기

- 엔티티 클래스(도메인)를 수정한다.
  - 현재 이름밖에 없다.
    - 나이, 성별 컬럼을 추가한다.
  - VS CODE의 MYSQL 확장 활용
    - 각 사용자의 정보를 db에서 직접 추가한다.
      - INSERT 문을 활용
        - 그림은 확장에서 삽입 후 SELECT 한 결과
  - 뷰를 수정한다.
    - testdb.html을 직접 수정 후 출력한다.
- 실습 결과 확인 – Q / A



id bigint	name varchar(255)	age	gender
1	홍길동	25	남
2	아저씨	40	남
3	김영희	30	여
4	이철수	22	남



User List

아이디	이름	나이	성별
1	홍길동	25	남
2	아저씨	40	남
3	김영희	30	여
4	이철수	22	남

Q & A

- 무엇이든 물어보세요!

